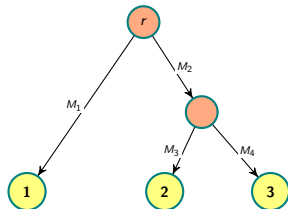


Phylogenetic models, Data Structures and Tools

Tobias Boege, **Antony Della Vecchia**,
Marina Garrote López, Benjamin Hollering

TU Berlin

ICMS 2026-07-22



OSCAR
SYMBOLIC TOOLS

Overview

- **Background** — graphical models, algebraic phylogenetics.
- **Implementation** — data structures and tools for working with these models in OSCAR.
- **Ongoing work.**

Graphical Models

- Fix a graph G .
- $\Theta \subseteq \mathbb{R}^d$ a semi algebraic set of parameters attached to G .
- Δ the space of **all distributions** on the observed variables.

$$\varphi : \Theta \rightarrow \mathcal{M}_G \subseteq \Delta$$

Graphical Models

- Fix a graph G .
- $\Theta \subseteq \mathbb{R}^d$ a semi algebraic set of parameters attached to G .
- Δ the space of **all distributions** on the observed variables.

$$\varphi : \Theta \rightarrow \mathcal{M}_G \subseteq \Delta$$

- Want to understand invariants that come from the Graph.

Graphical Models

- Fix a graph G .
- $\Theta \subseteq \mathbb{R}^d$ a semi algebraic set of parameters attached to G .
- Δ the space of **all distributions** on the observed variables.

$$\varphi : \Theta \rightarrow \mathcal{M}_G \subseteq \Delta$$

- Want to understand invariants that come from the Graph.
- In general many different types of graphs and statistical models.

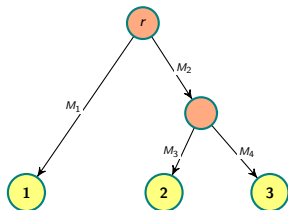
Graphical Models

- Fix a graph G .
- $\Theta \subseteq \mathbb{R}^d$ a semi algebraic set of parameters attached to G .
- Δ the space of **all distributions** on the observed variables.

$$\varphi : \Theta \rightarrow \mathcal{M}_G \subseteq \Delta$$

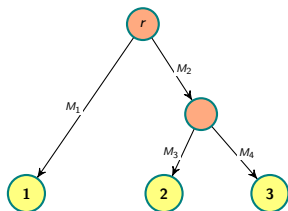
- Want to understand invariants that come from the Graph.
- In general many different types of graphs and statistical models.
- Use commutative algebra to understand $\overline{\mathcal{M}_G} = \overline{\varphi(\Theta)}$.

Algebraic Phylogenetics



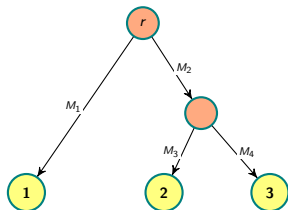
- Markov property: X_v depends only on ancestor and transition matrix M_e on that edge.

Algebraic Phylogenetics



- Markov property: X_v depends only on ancestor and transition matrix M_e on that edge.
- Want to find invariants in terms of the observables at the leaves.

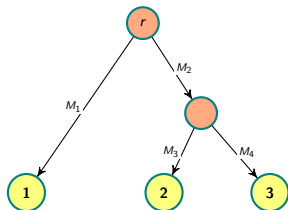
Algebraic Phylogenetics



- Markov property: X_v depends only on ancestor and transition matrix M_e on that edge.
- Want to find invariants in terms of the observables at the leaves.
- Hidden internal nodes $\Rightarrow$ marginalize:

$$p_{i_1 \dots i_n} = \sum_{x_v, v \text{ internal}} \pi(x_r) \prod_{u \rightarrow v} M_e(x_u, x_v).$$

Algebraic Phylogenetics

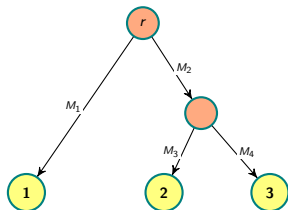


- Markov property: X_v depends only on ancestor and transition matrix M_e on that edge.
- Want to find invariants in terms of the observables at the leaves.
- Hidden internal nodes $\Rightarrow$ marginalize:

$$p_{i_1 \dots i_n} = \sum_{x_v, v \text{ internal}} \pi(x_r) \prod_{u \rightarrow v} M_e(x_u, x_v).$$

- Use invariants of the leaves to distinguish between topologies.

Algebraic Phylogenetics



- Markov property: X_v depends only on ancestor and transition matrix M_e on that edge.
- Want to find invariants in terms of the observables at the leaves.
- Hidden internal nodes $\Rightarrow$ marginalize:

$$p_{i_1 \dots i_n} = \sum_{x_v, v \text{ internal}} \pi(x_r) \prod_{u \rightarrow v} M_e(x_u, x_v).$$

- Use invariants of the leaves to distinguish between topologies.

The General Graphical Model Framework

```
abstract type GraphicalModel{T <: AbstractGraph, L <: Union{NamedTuple, Nothing}} end
```

The General Graphical Model Framework

```
abstract type GraphicalModel{T <: AbstractGraph, L <: Union{NamedTuple, Nothing}} end
```

$$\varphi^* : S \rightarrow R$$

Given implementations for

- `model_ring` (returns S coordinate ring of $\mathcal{M}$)
- `parameter_ring` (returns R coordinate ring of Θ)
- `parametrization` (returns φ^*)

We can write a generic `vanishing_ideal` implementation.

The General Graphical Model Framework

```
abstract type GraphicalModel{T <: AbstractGraph, L <: Union{NamedTuple, Nothing}} end
```

$$\varphi^* : S \rightarrow R$$

Given implementations for

- `model_ring` (returns S coordinate ring of $\mathcal{M}$)
- `parameter_ring` (returns R coordinate ring of Θ)
- `parametrization` (returns φ^*)

We can write a generic `vanishing_ideal` implementation.

- Set up the ideal $\langle s_i - \varphi^*(s_i) : s_i \in S \rangle$ for each generator s_i of the model ring S ,
- then eliminate the variables of the parameter ring R .

Constructing a PhylogeneticModel

```
abstract type GraphicalModel{T <: AbstractGraph, L <: Union{NamedTuple, Nothing}} end

struct PhylogeneticModel{T, L, M, R} <: GraphicalModel{T, L}
  graph::T
  labelings::L
  transition_matrices_structure::Matrix{M}
  root_distribution::Vector{R}
  n_states::Int
end
```

Constructing a PhylogeneticModel

```
abstract type GraphicalModel{T <: AbstractGraph, L <: Union{NamedTuple, Nothing}} end

struct PhylogeneticModel{T, L, M, R} <: GraphicalModel{T, L}
    graph::T
    labelings::L
    transition_matrices_structure::Matrix{M}
    root_distribution::Vector{R}
    n_states::Int
end
```

Multiple ways to construct models

```
julia> M = [:a :b; :b :a];
julia> phylo_t = phylogenetic_tree("((H:3,(C:1,B:1):2):1,G:4);");
julia> PM1 = phylogenetic_model(phylo_t, M);
julia> typeof(PM1)
PhylogeneticModel{PhylogeneticTree{QQFieldElem}, ...}

julia> G = graph_from_edges(Directed, [[5,1], [6,5], [7,6], [7,8],
                                         [8,5], [6,2], [7,3], [8,4]]);
julia> PM2 = phylogenetic_model(G, M);
julia> typeof(PM2)
PhylogeneticModel{PhylogeneticNetwork{1, 1}, ... }
```


Classical Biological Models

Common **linear constraints** on Θ :

$$\underbrace{\begin{pmatrix} a & b & b & b \\ b & a & b & b \\ b & b & a & b \\ b & b & b & a \end{pmatrix}}_{\text{Jukes-Cantor}}$$

$$\underbrace{\begin{pmatrix} a & b & c & b \\ b & a & b & c \\ c & b & a & b \\ b & c & b & a \end{pmatrix}}_{\text{Kimura 2}}$$

$$\underbrace{\begin{pmatrix} a & b & c & d \\ b & a & d & c \\ c & d & a & b \\ d & c & b & a \end{pmatrix}}_{\text{Kimura 3}}$$

Classical Biological Models

Common **linear constraints** on Θ :

$$\underbrace{\begin{pmatrix} a & b & b & b \\ b & a & b & b \\ b & b & a & b \\ b & b & b & a \end{pmatrix}}_{\text{Jukes-Cantor}}$$

$$\underbrace{\begin{pmatrix} a & b & c & b \\ b & a & b & c \\ c & b & a & b \\ b & c & b & a \end{pmatrix}}_{\text{Kimura 2}}$$

$$\underbrace{\begin{pmatrix} a & b & c & d \\ b & a & d & c \\ c & d & a & b \\ d & c & b & a \end{pmatrix}}_{\text{Kimura 3}}$$

Available in OSCAR:

- group-based: `jukes_cantor_model`, `kimura2_model`, `kimura3_model`;
- `general_markov_model`, `general_time_reversible_model`.

Group-Based Models: Fourier $\Rightarrow$ Toric

Fourier transform diagonalizes every transition matrix.

- Probability coords $p \longrightarrow$ **Fourier coords** q .
- Parametrization $\rightarrow$ **monomial map**.
- $\Rightarrow$ **toric variety** — uses 4t12 to first find Markov basis.

Group-Based Models: Fourier $\Rightarrow$ Toric

Fourier transform diagonalizes every transition matrix.

- Probability coords $p \rightarrow$ **Fourier coords** q .
- Parametrization $\rightarrow$ **monomial map**.
- $\Rightarrow$ **toric variety** — uses 4ti2 to first find Markov basis.

```
julia> T = graph_from_edges(Directed, [[1, 2], [1, 3], [1, 4]]);  
julia> GM = jukes_cantor_model(T)  
Group-based phylogenetic model on a tree with 3 leaves and 3 edges  
with root distribution [1//4, 1//4, 1//4, 1//4],  
transition matrices of the form
```

```
[ :a :b :b :b;  
  :b :a :b :b;  
  :b :b :a :b;  
  :b :b :b :a]
```

and fourier parameters of the form $[:x, :y, :y, :y]$.

```
julia> vanishing_ideal(GM)  
Ideal generated by  
-q[2,3,4]^2*q[1,1,1] + q[2,2,1]*q[2,1,2]*q[1,2,2]
```

```
julia> PM = phylogenetic_model(GM)  
Phylogenetic model on a tree with 3 leaves and 3 edges
```

```
...  
julia> vanishing_ideal(PM)
```

```
Ideal generated by  
2*p[1,2,3]^3 - p[1,2,3]^2*p[1,2,2] - p[1,2,3]^2*p[1,2,1] - p[1,2,3]^2*p[1,1,2] +  
p[1,2,3]^2*p[1,1,1] - p[1,2,3]*p[1,2,2]^2 - p[1,2,3]*p[1,2,1]^2 - p[1,2,3]*p[1,1,2]^2 +  
p[1,2,3]*p[1,1,1]^2 + p[1,2,2]^2*p[1,2,1] + p[1,2,2]^2*p[1,1,2] + ...
```

The `mrdr` File Format [DV-Joswig-Lorenz 24]

- Based on JSON (Human readable).
- Generalizes ideas from the `polymake` format to algebraic data.
- First version developed using OSCAR.
- Implementations for other systems like Macaulay2, Magma and SageMath
- All types in the Algebraic Statistic section of OSCAR can be stored using the format.

Multigraded Implicitization [Cummings-Hollering 26]

Problem: Generators of vanishing ideals can become very difficult to compute (Gröbner basis).

Multigraded Implicitization [Cummings-Hollering 26]

Problem: Generators of vanishing ideals can become very difficult to compute (Gröbner basis).

Ideal is homogeneous in a **multigrading**, this provides an alternative to finding generators up to a given degree.

```
julia> M = kimura3_model(N1);  
julia> phi = parametrization(M);  
julia> comps = components_of_kernel(3, phi);
```

Multigraded Implicitization [Cummings-Hollering 26]

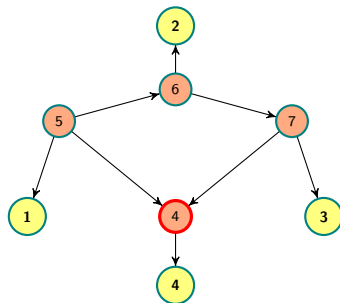
Problem: Generators of vanishing ideals can become very difficult to compute (Gröbner basis).

Ideal is homogeneous in a **multigrading**, this provides an alternative to finding generators up to a given degree.

```
julia> M = kimura3_model(N1);  
julia> phi = parametrization(M);  
julia> comps = components_of_kernel(3, phi);
```

Leaves	Deg	Macaulay2 (s)	OSCAR (s)
4	2	0.472	0.050
4	3	117	1.06
5	2	329	0.951
5	3	—	372

Kimura 3 on sunlet networks.

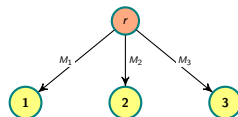


4-leaf sunlet network

Parallel Computation

Parallel:

- OSCAR has support for multi process computations
- degree-5 general Markov, 3-leaf tree
- 32 processes, 150 GB;
- < 3 hours vs. > 24 hours (serial).
- Results available on Zenodo
<https://zenodo.org/records/18312720>



3-leaf tree

Ongoing work

Results outlive the computation — and the software.

- OSCAR types → `mrda` format (JSON-based).
- `mrda` files → **MongoDB** database: `OscarDB`.
- All phylo models from <https://algebraicphylogenetics.org>.
- Plans to provide tighter integration,
- and expand to include networks and other model types.

Ongoing work

Results outlive the computation — and the software.

- OSCAR types → `mrda` format (JSON-based).
- `mrda` files → **MongoDB** database: `OscarDB`.
- All phylo models from <https://algebraicphylogenetics.org>.
- Plans to provide tighter integration,
- and expand to include networks and other model types.

Thank you!